ENTERPRISE DEVOPS

MASTER NOTES

Kubernetes · Azure · AKS · Docker · CI/CD

Production-Grade · Enterprise Level · Portfolio Ready

STAGE 1	STAGE 2	STAGE 3	STAGE 4	STAGE 5	STAGE 6–8
Docker	K8s Local	GitHub CI	Azure DevOps	AKS	E2E + Prod

STAGE 1 · DOCKER MASTERY

Stage 1 — Docker Mastery

Key Concepts

- Multi-stage builds — separate builder from final image
- Non-root users — always create appuser, never run as root
- Layer caching — COPY package*.json BEFORE source code
- Image tagging strategy — semver + git SHA (e.g. 1.0.0-abc1234)
- Image size goal — under 60MB using alpine + multi-stage
- .dockerignore — exclude node_modules, .git, .env files

Production Dockerfile Pattern

```
# Stage 1: Builder
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

# Stage 2: Production
FROM node:20-alpine AS production
RUN addgroup -g 1001 -S appgroup && \
    adduser  -u 1001 -S appuser -G appgroup
WORKDIR /app
COPY --from=builder --chown=appuser:appgroup /app/node_modules ./node_modules
COPY --chown=appuser:appgroup app.js ./
ENV NODE_ENV=production
EXPOSE 3000
USER appuser
HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "app.js"]
```

Build & Tag Commands

```
export GIT_SHA=$(git rev-parse --short HEAD)
export APP_VERSION=1.0.0
export IMAGE=youruser/devops-api

docker build \
  -t $IMAGE:latest \
  -t $IMAGE:$APP_VERSION \
  -t $IMAGE:$APP_VERSION-$GIT_SHA .

docker push $IMAGE:$APP_VERSION-$GIT_SHA
```

Push to Azure Container Registry (ACR)

```
az group create --name devops-rg --location eastus
az acr create --resource-group devops-rg \
  --name devopsacr2024 --sku Standard --admin-enabled false

az acr login --name devopsacr2024
export ACR=$(az acr show --name devopsacr2024 --query loginServer -o tsv)

docker tag $IMAGE:$APP_VERSION-$GIT_SHA $ACR/devops-api:$APP_VERSION-$GIT_SHA
docker push $ACR/devops-api:$APP_VERSION-$GIT_SHA
```

Image Size Comparison

```
node:20 (base)          →  ~1.1 GB ❌
node:20-slim            →  ~240 MB ⚠️
node:20-alpine          →  ~180 MB ⚠️
Multi-stage alpine      →   ~55 MB ✅
```

Common Docker Mistakes

- ❌ COPY . . before npm install — breaks layer cache every code change
- ❌ Running as root — security risk
- ❌ npm install instead of npm ci — non-deterministic
- ❌ No .dockerignore — leaks secrets, huge context
- ✅ FIX: COPY package*.json first → npm ci → COPY src

Interview Angle — Docker

"I use multi-stage builds — builder installs deps, production copies only runtime. Images stay under 60MB vs 1GB+. Always non-root, npm ci for deterministic builds, tag with semver + git SHA so every image traces to a commit. ACR Standard SKU with lifecycle policies auto-purge images older than 90 days."

STAGE 2 · LOCAL KUBERNETES (MINIKUBE / KIND)

Stage 2 — Local Kubernetes Practice

Cluster Setup

```
minikube start --driver=docker --cpus=4 --memory=8g \
  --kubernetes-version=v1.29.0
minikube addons enable ingress
minikube addons enable metrics-server
kubectl get nodes
```

Essential Kubernetes Objects — Quick Reference

Object	Purpose / Key Setting
Deployment	Manages ReplicaSets; rolling update strategy
Service (ClusterIP)	Internal pod-to-pod communication
Service (NodePort)	Expose to external on static port 30000-32767
ConfigMap	Non-sensitive config (env vars, files)
Secret	Sensitive data (base64 encoded, use Vault in prod)
HPA	Auto-scale pods on CPU/memory metrics
PDB	Guarantee minimum pods during disruptions
Ingress	HTTP routing, TLS termination (NGINX controller)
LimitRange	Default resource limits per namespace
NetworkPolicy	Pod-level firewall rules

Key Deployment YAML Fields

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1           # +1 extra during update
    maxUnavailable: 0     # never go below desired replicas

resources:
  requests: { memory: 128Mi, cpu: 100m }
  limits:   { memory: 256Mi, cpu: 500m }

livenessProbe:            # restart if unhealthy
  httpGet: { path: /health, port: 3000 }
  initialDelaySeconds: 15
```

```
  periodSeconds: 20
  failureThreshold: 3

readinessProbe:          # route traffic only when ready
  httpGet: { path: /health, port: 3000 }
  initialDelaySeconds: 5
  periodSeconds: 10

securityContext:
  runAsNonRoot: true
  runAsUser: 1001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
```

HPA Configuration

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: devops-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: { type: Utilization, averageUtilization: 70 }
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300  # wait 5min before down-scale
```

Troubleshooting Commands

Failure	Debug Commands
CrashLoopBackOff	kubectl logs <pod> --previouskubectl describe pod <pod>
OOMKilled	kubectl describe pod | grep OOMkubectl top pods
ImagePullBackOff	kubectl describe pod → check EventsVerify ACR attached to AKS
HPA not scaling	kubectl top nodes (metrics-server?)kubectl describe hpa
Pending pod	kubectl describe pod → Insufficient cpu/memoryCheck resource requests vs node capacity

Rollback Commands

```
kubectl rollout history deployment/devops-api -n devops-app
kubectl rollout undo deployment/devops-api -n devops-app
kubectl rollout undo deployment/devops-api --to-revision=2 -n devops-app
kubectl rollout status deployment/devops-api -n devops-app
```

STAGE 3 · CI PIPELINE — GITHUB ACTIONS

Stage 3 — GitHub Actions CI Pipeline

Pipeline Stages

- 1. Code Checkout
- 2. Install & Unit Test
- 3. Security Scan (Trivy image + fs scan, npm audit)
- 4. Build & Push multi-arch image to Docker Hub

Secrets Required

```
# GitHub → Settings → Secrets → Actions:
DOCKERHUB_USERNAME
DOCKERHUB_TOKEN    # Docker Hub → Account Settings → Security
SNYK_TOKEN         # optional: snyk.io
```

GitHub Actions YAML — Key Sections

```
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: '20', cache: 'npm' }
    - run: npm ci && npm test

  security-scan:
    needs: build-and-test
    steps:
    - name: Trivy Image Scan
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: myimage:tag
        exit-code: '1'           # FAIL on findings
        severity: 'HIGH,CRITICAL'
        ignore-unfixed: true

  push-image:
    needs: [build-and-test, security-scan]
    if: github.ref == 'refs/heads/main'
    steps:
    - uses: docker/build-push-action@v5
```

```
      with:
        push: true
        platforms: linux/amd64,linux/arm64
        tags: |
          myuser/devops-api:latest
          myuser/devops-api:1.0.0-${{ env.GIT_SHA }}
        cache-from: type=gha
        cache-to: type=gha,mode=max
```

Security Scan Logic

- Trivy image scan: scans installed packages inside Docker image
- Trivy fs scan: scans source code dependencies (package.json, requirements.txt)
- npm audit: Node.js specific CVE database check
- exit-code: 1 makes pipeline FAIL on HIGH/CRITICAL — mandatory for enterprise
- ignore-unfixed: true skips vulns with no available fix (reduces noise)

STAGE 4 · AZURE DEVOPS — ENTERPRISE CI/CD

Stage 4 — Azure DevOps (Main Focus)

Core Azure DevOps Concepts

Concept	What It Does
Service Connection	Authenticates pipeline to ACR, Azure, K8s
Variable Group	Shared secrets/vars across pipelines (Library)
Environment	Target (dev/staging/prod) with approval gates
Approval Gate	Human gate — blocks pipeline until approved
Artifact	Build output passed between stages
Agent Pool	VMs that run your pipeline jobs
Deployment Job	Special job that tracks environment deployments

Setup Steps

- 1. Create Azure DevOps Project
- 2. Create Service Connections (ACR, Azure subscription, Kubernetes)
- 3. Create Variable Groups (production-secrets, dev-secrets)
- 4. Create Environments (development, staging, production)
- 5. Add Approval Gates to staging (1 approver) and production (2 approvers)

Variable Groups — Secret Handling

```
# In azure-pipelines.yml:
variables:
  - group: production-secrets   # pulls from Library
  - name: imageRepository
    value: devops-api

# Variable Group contains:
# ACR_NAME          = devopsacr2024      (not secret)
# ACR_LOGIN_SERVER  = *.azurecr.io       (not secret)
# DB_PASSWORD       = ***                (🔒 secret - masked in logs)
# API_KEY           = ***                (🔒 secret)
```

azure-pipelines.yml — Pipeline Stages

```
stages:
```

```
- stage: Build
  jobs:
  - job: BuildJob
    steps:
    - task: NodeTool@0
      inputs: { versionSpec: '20.x' }
    - script: npm ci

- stage: Test
  dependsOn: Build
  jobs:
  - job: TestJob
    steps:
    - script: npm test -- --ci --coverage
    - task: PublishTestResults@2

- stage: SecurityScan
  dependsOn: Test
  jobs:
  - job: TrivyScan
    steps:
    - script: |
        trivy image --exit-code 1 \
          --severity HIGH,CRITICAL \
          --ignore-unfixed \
          $(imageRepository):scan-$(Build.BuildId)

- stage: PushToACR
  dependsOn: SecurityScan
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
  jobs:
  - job: PushJob
    steps:
    - task: Docker@2
      inputs:
        command: buildAndPush
        containerRegistry: acr-service-connection
        tags: |
          $(Build.BuildId)
          $(APP_VERSION)-$(Build.SourceVersion)

- stage: DeployStaging
  dependsOn: PushToACR
  jobs:
  - deployment: DeployStaging
    environment: staging     # triggers approval gate
    strategy:
      runOnce:
        deploy:
          steps:
          - task: KubernetesManifest@0
            inputs:
              action: deploy
              namespace: devops-app

- stage: DeployProduction
  dependsOn: DeployStaging
  jobs:
  - deployment: DeployProd
    environment: production  # 2 approvers required
```

Azure DevOps Best Practices

- Never hardcode secrets — always use Variable Groups marked as secret
- Use condition: succeeded() to fail fast — don't deploy if tests fail
- Use deployment jobs (not regular jobs) for environment tracking
- Separate Variable Groups per environment (dev-secrets, prod-secrets)
- Set pipeline permissions — restrict who can edit pipelines
- Enable branch policies — require PR + CI pass before merge to main

STAGE 5 · AKS — AZURE KUBERNETES SERVICE

Stage 5 — AKS Production Implementation

AKS Cluster Creation

```
export RG=aks-prod-rg
export CLUSTER=aks-prod-cluster
export ACR=devopsacr2024

# Resource Group
az group create --name $RG --location eastus

# VNet (required for Azure CNI)
az network vnet create --resource-group $RG \
  --name aks-prod-vnet \
  --address-prefix 10.0.0.0/8 \
  --subnet-name aks-subnet \
  --subnet-prefix 10.240.0.0/16

export SUBNET_ID=$(az network vnet subnet show \
  -g $RG --vnet-name aks-prod-vnet \
  --name aks-subnet --query id -o tsv)

# Create AKS Cluster
az aks create \
  --resource-group $RG --name $CLUSTER \
  --node-count 3 --min-count 2 --max-count 10 \
  --enable-cluster-autoscaler \
  --node-vm-size Standard_DS3_v2 \
  --network-plugin azure \
  --vnet-subnet-id $SUBNET_ID \
  --enable-managed-identity \
  --enable-addons monitoring \
  --zones 1 2 3 \
  --tier Standard

# Attach ACR (enables pull without pull secrets)
az aks update -g $RG -n $CLUSTER --attach-acr $ACR

# Get credentials
az aks get-credentials -g $RG -n $CLUSTER
```

NGINX Ingress Controller on AKS

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update

helm install ingress-nginx ingress-nginx/ingress-nginx \
  --create-namespace --namespace ingress-nginx \
  --set controller.replicaCount=2 \
  --set controller.nodeSelector."kubernetes\.io/os"=linux

kubectl get svc ingress-nginx-controller -n ingress-nginx -w
```

```
# Wait for EXTERNAL-IP from Azure Load Balancer
```

AKS-Specific Troubleshooting

Issue	Root Cause & Fix
Subnet IP exhaustion	Azure CNI = 1 IP per pod. Expand subnet CIDR or add new subnet node pool
ACR ImagePullBackOff	az aks update --attach-acr $ACR_NAME -g $RG -n $CLUSTER
Node NotReady	SSH via debug pod: kubectl debug node/<name> — check kubelet status
Ingress 502	Check endpoints (kubectl get endpoints) — readiness probe failing?
Autoscaler not triggering	Check node has pending pods AND requests set — HPA needs metrics-server

AKS Architecture — Multi-Zone

```
Users → Azure AppGateway (WAF) → NGINX Ingress → AKS Pods
```

- System node pool: coredns, metrics-server, ingress-nginx
- App node pool: your workloads, autoscale 3→10
- Spot node pool: batch jobs (80% cheaper, evictable)
- Zones 1-2-3: pods spread across AZs via topologySpreadConstraints

STAGE 6 · HELM + KEY VAULT + FULL E2E PIPELINE

Stage 6 — Full Enterprise E2E

Helm Chart Structure

```
devops-api/
├── Chart.yaml
├── values.yaml           # base defaults
├── values-dev.yaml       # dev overrides
├── values-staging.yaml
├── values-prod.yaml      # prod overrides (more replicas, limits)
└── templates/
    ├── deployment.yaml
    ├── service.yaml
    ├── ingress.yaml
    ├── hpa.yaml
    └── _helpers.tpl
```

Helm Deploy Commands

```
# First deploy
helm install devops-api ./devops-api \
  --namespace production --create-namespace \
  -f values-prod.yaml \
  --set image.tag=1.0.0-abc1234

# Upgrade (with auto-rollback on failure)
helm upgrade devops-api ./devops-api \
  --namespace production \
  -f values-prod.yaml \
  --set image.tag=1.1.0-def5678 \
  --atomic \        # rollback if fails
  --cleanup-on-fail \
  --wait \
  --timeout 5m0s

# Rollback
helm rollback devops-api 1 --namespace production
helm history devops-api --namespace production
```

Azure Key Vault Integration

```
# Enable addon
az aks enable-addons --addons azure-keyvault-secrets-provider \
  --name $CLUSTER --resource-group $RG

# Create Key Vault
az keyvault create --name aks-prod-kv \
  --resource-group $RG --enable-rbac-authorization true

# Store secrets
az keyvault secret set --vault-name aks-prod-kv \
```

```
  --name DB-PASSWORD --value prod-secure-pass

# Grant AKS Managed Identity access
export MI=$(az aks show -n $CLUSTER -g $RG \
  --query addonProfiles.azureKeyvaultSecretsProvider.identity.clientId -o tsv)

az role assignment create \
  --role 'Key Vault Secrets User' \
  --assignee $MI \
  --scope $(az keyvault show --name aks-prod-kv --query id -o tsv)
```

SecretProviderClass YAML

```
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: azure-kvs-provider
  namespace: production
spec:
  provider: azure
  secretObjects:
  - secretName: app-secrets
    type: Opaque
    data:
    - objectName: DB-PASSWORD
      key: db-password
  parameters:
    useVMManagedIdentity: 'true'
    userAssignedIdentityID: '<MI_CLIENT_ID>'
    keyvaultName: aks-prod-kv
    objects: |
      array:
        - |
          objectName: DB-PASSWORD
          objectType: secret
    tenantId: '<TENANT_ID>'
```

STAGE 7 · PRODUCTION TROUBLESHOOTING PLAYBOOK

Stage 7 — Troubleshooting Reference

CrashLoopBackOff

```
kubectl get pods -n <ns>
kubectl describe pod <pod> -n <ns>    # check Events
kubectl logs <pod> -n <ns> --previous # logs before crash
kubectl exec -it <pod> -- /bin/sh     # manual inspect
```

Root Causes: App crash on startup · Wrong CMD · Missing env var · DB unreachable · Liveness probe fires too early (fix: increase initialDelaySeconds)

OOMKilled

```
kubectl describe pod <pod> | grep -A5 'Last State'
# OOMKilled, Exit Code: 137
kubectl top pods -n <ns>

# Fix: increase memory limit
kubectl set resources deployment devops-api \
  --limits=memory=512Mi -n <ns>
```

Root Causes: Memory leak in app · Limit set too low · Sudden traffic spike · Missing connection pool cleanup

ImagePullBackOff

```
kubectl describe pod <pod>   # check Events for 'unauthorized'

# Minikube fix: use minikube's docker daemon
eval $(minikube docker-env)
docker build -t myimage:v1 .

# AKS fix: attach ACR
az aks update --attach-acr $ACR -g $RG -n $CLUSTER
```

Ingress 502 Bad Gateway — Step by Step

```
# Step 1: Check service exists
kubectl get svc -n <ns>

# Step 2: Are endpoints registered? (empty = readiness failing)
kubectl get endpoints <service> -n <ns>

# Step 3: Can you hit the pod directly?
kubectl port-forward pod/<pod> 8080:3000 -n <ns>
curl localhost:8080/health
```

```
# Step 4: Check ingress config
kubectl describe ingress <name> -n <ns>

# Step 5: Ingress controller logs
kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx
```

Node NotReady

```
kubectl describe node <node-name>    # check Conditions

# Debug via privileged pod
kubectl debug node/<node-name> -it --image=ubuntu -- bash
chroot /host
systemctl status kubelet
journalctl -u kubelet -n 50

# Before maintenance: cordon + drain
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
```

HPA Not Scaling

```
kubectl top nodes   # if fails: metrics-server not installed
kubectl describe hpa devops-api-hpa -n <ns>
# 'unknown/70%' = metrics not available yet

# Fix: enable metrics-server
minikube addons enable metrics-server

# Load test to trigger HPA
kubectl run load --image=busybox:1.28 --rm -it -- \
  /bin/sh -c 'while sleep 0.01; do wget -q -O- http://svc/api; done'

kubectl get hpa -n <ns> -w   # watch scale events
```

STAGE 8 · PRODUCTION ARCHITECTURE DESIGN

Stage 8 — Production Architecture

Full Architecture Stack

```
Internet
   │
   ▼
[Azure DDoS Protection]
   │
   ▼
[Azure Front Door / CDN]    ← static assets, global routing
   │
   ▼
[Azure Application Gateway v2]  ← WAF, SSL termination
   │
   ▼
[AKS Cluster — Multi-Zone (1, 2, 3)]
  ┌──────────────────────────────────────┐
  │  System Pool:  coredns, metrics-server │
  │                ingress-nginx           │
  │                                        │
  │  App Pool:     api-pods (autoscale 3-10) │
  │                                        │
  │  Spot Pool:    batch / non-critical    │
  └──────────────────────────────────────┘
   │            │
   ▼            ▼
[Redis Cache] [Azure SQL / CosmosDB]

Supporting:
[ACR] [Key Vault] [Azure Monitor] [Managed Identity]
```

HA & Resilience Checklist

- topologySpreadConstraints — spread pods across AZs and nodes
- PodDisruptionBudget — minAvailable: 2, protect during node drains
- Cluster Autoscaler — scale node pool 2→10 on pending pods
- HPA — scale pods on CPU 70% / memory 80%
- ReadinessProbe — traffic only to ready pods
- --atomic in helm upgrade — auto rollback on deploy failure
- Multi-zone AKS — --zones 1 2 3 at cluster creation

Cost Optimisation

- Spot node pool — 80% cheaper, use for batch, stateless workloads
- Cluster autoscaler scales to 0 on spot pool at night
- Right-size requests — use kubectl-resource-capacity to find waste

- ACR lifecycle policy — auto-delete untagged images > 30 days
- Reserved Instances on system node pool — 40-60% savings
- Azure Dev/Test pricing for non-production clusters

Security Hardening

- Managed Identity — no static credentials anywhere
- Key Vault CSI — secrets mounted, never in YAML
- readOnlyRootFilesystem: true on all containers
- NetworkPolicy — default deny, explicit allow per service
- ACR — admin-enabled: false, use service principal/MI
- Azure Policy — enforce resource limits, disallow privileged containers
- Private cluster — AKS API server not publicly accessible

STAGE 9 · INTERVIEW PREPARATION

Interview Q&A — Top 30

Docker Questions

Question	Key Answer Points
Walk me through your production Dockerfile	Multi-stage, non-root, npm ci, <60MB, tagged with git SHA
Why multi-stage build?	Builder has dev tools, final image has only runtime = smaller, more secure
npm install vs npm ci?	npm ci is deterministic, faster, uses exact package-lock.json
How do you reduce image size?	Alpine base, multi-stage, .dockerignore, no dev deps in final stage

Kubernetes Questions

Question	Key Answer Points
Liveness vs Readiness probe?	Liveness = restart if unhealthy; Readiness = remove from LB if not ready
Rolling update vs blue-green?	Rolling = replace pods one by one. Blue-green = full parallel env, instant cutover
How to debug CrashLoopBackOff?	kubectl logs --previous, kubectl describe pod, check exit code
What is PDB?	PodDisruptionBudget ensures min pods stay up during voluntary disruptions (drains, upgrades)
Pod is Pending — why?	Insufficient cpu/memory, taint/toleration mismatch, affinity, PVC not bound
How does HPA work?	Queries metrics-server every 15s, scales if avg CPU/memory > threshold

CI/CD & Azure DevOps Questions

Question	Key Answer Points
How do you handle secrets in pipeline?	Variable Groups (encrypted), never in YAML. Runtime: Key Vault + Managed Identity

What is a deployment job vs regular job?	Deployment job tracks which version deployed to which environment, enables approval gates
How do you prevent bad deploys to prod?	Security scan gate, staging approval, production 2-approver gate, --atomic in Helm
Azure DevOps vs GitHub Actions?	AzDO: enterprise RBAC, approval gates, Variable Groups. GHA: simpler, OSS-friendly

AKS Questions

Question	Key Answer Points
Azure CNI vs kubenet?	Azure CNI: pods get VNet IPs, direct routing, needed for ACI/AppGW integration. Kubenet: NAT, simpler
Why Azure CNI subnet exhaustion?	1 IP per pod. Solution: larger subnet or add new subnet node pool
How do you attach ACR to AKS?	az aks update --attach-acr — grants AcrPull role to kubelet managed identity
How to do zero-downtime deploy?	maxUnavailable: 0, readiness probe, terminationGracePeriod, preStop hook

STAR Answer Framework

How to Answer Any Technical Question

SITUATION → ACTION → RESULT → LEARNINGS

Example: 'Tell me about a production incident you resolved'

Situation: Pods were OOMKilled every 2h during peak traffic

Action: kubectl top pods showed 400MB usage vs 256MB limit. Heap profiling revealed Redis connection pool not releasing on timeout

Result: Fixed connection pooling, increased limits to 512MB, added Azure Monitor alert at 80%. Zero OOMKills for 3 weeks

Learnings: Now standard to set limits at 2x p95 baseline, not guesses. Added memory dashboards to all services at launch

STAGE 10 · HIGH-SALARY ENGINEER GUIDE

High-Salary Engineer — What Differentiates You

₹6 LPA vs ₹16+ LPA Engineer

₹6 LPA Engineer	₹16+ LPA Engineer
Follows tutorials	Designs architecture
Runs kubectl commands	Writes production Helm charts
Fixes issues reactively	Prevents issues proactively
Knows commands	Understands WHY each decision
One-environment deploys	Multi-env pipelines with approvals
No security layer	Trivy + Key Vault + RBAC integrated
'It works locally'	HPA + PDB + probes + monitoring
Can't explain tradeoffs	Azure CNI vs kubenet? Knows both
YAML copied from docs	Templated with Helm, parameterised
No observability setup	Dashboards + alerts at launch

Resume Bullet Points That Get Interviews

❌ WEAK (avoid this)

"Worked on Kubernetes deployments and Docker containers"

✅ STRONG (use these)

- "Architected enterprise CI/CD pipeline (Azure DevOps) reducing deployment time from 45min to 8min with multi-stage pipelines, Trivy security gates, and automated Helm rollouts to AKS"
- "Designed AKS production cluster (Azure CNI, multi-zone, 3-10 autoscaling) with Key Vault CSI integration, eliminating all static credentials from workload manifests"
- "Reduced Docker image size from 1.1GB to 54MB via multi-stage builds; implemented non-root containers and read-only filesystems across 12 microservices"
- "Built HPA + cluster autoscaler handling 10x traffic spikes with zero manual intervention, saving ~₹40k/month on over-provisioned VMs"

Your 90-Day Plan

Timeframe	Goal
Week 1-2	Stage 1 complete — Docker multi-stage + ACR. Screenshot everything
Week 3-4	Stage 2 complete — Minikube cluster, all YAMLs working, simulated failures
Week 5-6	Stage 3 complete — GitHub Actions CI with Trivy passing green
Week 7-9	Stage 4 complete — Azure DevOps full pipeline, approvals, environments
Week 10-12	Stage 5+6 — AKS + Helm + Key Vault + architecture diagram

Final Portfolio Checklist

- GitHub: 6 repos (one per project), clean README + architecture diagrams
- LinkedIn: post architecture diagram with explanation of decisions
- Resume: 3-4 bullet points per project with measurable impact
- Can whiteboard any diagram and explain every tradeoff

Skills That Earn ₹16+ LPA

- ✅ Azure DevOps multi-stage pipelines with approval gates
- ✅ AKS with Azure CNI, multi-zone, cluster autoscaler
- ✅ Helm-based deployments with per-environment values
- ✅ Azure Key Vault + Managed Identity (zero static creds)
- ✅ Trivy security gates in pipelines
- ✅ HPA + PDB + rolling updates (zero-downtime)
- ✅ Troubleshoot under pressure with structured approach
- ✅ Design + justify architecture decisions with tradeoffs

STAGE 11 · QUICK REFERENCE CHEATSHEET

Quick Reference — Commands Cheatsheet

Docker

```
docker build -t image:tag .
docker push image:tag
docker images / docker ps -a
docker logs <container> -f
docker exec -it <container> /bin/sh
docker stats
docker system prune -af
```

kubectl — Daily Use

```
kubectl get pods/deploy/svc/ingress -n <ns>
kubectl describe pod/deploy/node <name> -n <ns>
kubectl logs <pod> -n <ns> --previous -f
kubectl exec -it <pod> -n <ns> -- /bin/sh
kubectl top pods/nodes
kubectl rollout status/history/undo deploy/<name> -n <ns>
kubectl scale deploy/<name> --replicas=5 -n <ns>
kubectl apply -f manifest.yaml
kubectl delete pod <name> -n <ns>         # force reschedule
kubectl get events -n <ns> --sort-by='.lastTimestamp'
kubectl port-forward pod/<name> 8080:3000 -n <ns>
```

Helm

```
helm install <name> ./chart -n <ns> -f values-prod.yaml \
  --set image.tag=v1
helm upgrade <name> ./chart --atomic --wait --timeout 5m0s
helm rollback <name> <revision> -n <ns>
helm history <name> -n <ns>
helm list -A
helm template ./chart -f values.yaml
helm uninstall <name> -n <ns>
```

az — AKS

```
az aks get-credentials -g $RG -n $CLUSTER
az aks nodepool list -g $RG --cluster-name $CLUSTER -o table
az aks update -g $RG -n $CLUSTER --attach-acr $ACR
az acr repository list --name $ACR --output table
az acr repository show-tags --name $ACR --repository devops-api
az monitor log-analytics query --workspace $WS_ID \
  --analytics-query 'ContainerLog | limit 50'
```

Azure DevOps Built-in Variable	Value
$(Build.BuildId)	Unique integer build ID
$(Build.SourceBranch)	refs/heads/main
$(Build.SourceVersion)	Full git commit SHA
$(Build.Repository.Name)	Repo name
$(System.DefaultWorkingDirectory)	Agent workspace path
$(Pipeline.Workspace)	Artifact download location